layout is very close to pure JavaScript and also similar to the code representing dynamic points of interest (POIs) on a D3.js map. Not only are the methods the same, but the very data could be the same, formulated in one way for lists and paragraphs and spans, while formulated in another way for geospatial representation. The class of data-driven documents is already broad and becomes even more all-encompassing when you also treat images and text as data.

### 1.2.2 D3 is about selecting and binding

Throughout this chapter, you'll see code snippets that you can run in your browser to make changes to the graphical appearance of elements on your website. At the end of the chapter is an application written in D3 that explains the basics of the code we're running in JavaScript. But before that we'll explore the principles of web development using D3, and you'll see this pattern of code over and over again: selecting.

Imagine we have a set of data, such as the price and size of a few houses, and a set of web page elements, whether graphics or traditional `<div>` elements, and that we want to represent the dataset, whether with text or through size and color. A *selection* is the group of all of them together, and we perform actions on the elements in the group, such as moving them, changing their color, or updating the values in the data. We work with the data and the web page elements separately, but the real power of D3 comes from using selections to combine data and web page elements.

Here's a selection without any data:

```
d3.selectAll("circle.a").style("fill", "red").attr("cx", 100);
```

This takes every circle on our page with the class of `"a"` and turns it red and moves it so that its center is 100 pixels to the right of the left side of our `<svg>` canvas. Likewise, this code turns every div on our web page red and changes its class to `"b"`:

```
d3.selectAll("div").style("background", "red").attr("class", "b");
```

But before we can change our circles and divs, we'll need to create them, and before we do that, it's best to understand what's happening in this pattern.

The first part of that line of code, `d3.selectAll()`, is part of the core functionality necessary for understanding D3: selections. Selections can be made with `d3.select()`, which selects the first single element found, but more often you'll use `d3.selectAll()`, which can be used to select multiple elements. Selections are a group of one or more web page elements that may be associated with a set of data, like the following code, which binds the elements in the array [1,5,11,3] to `<div>` elements with the class of `"market"`:

```
d3.selectAll("div.market").data([1,5,11,3])
```

This association is known in D3 as *binding data,* and you can think of a selection as a set of web page elements and a corresponding, associated set of data. Sometimes there are more data elements than DOM elements, or vice versa, in which case D3 has